HUMAN RESOURCE MANAGEMENT SYSTEM

Research, Feasibility Check and Proposed Implementation

Prepared by
Naveen Kumar S

1. Introduction

A Human Resource Management System (HRMS) is a software system used to organize employee-related information and routine HR activities in one place. Instead of maintaining employee details, attendance records and leave information separately, an HRMS connects these activities through a common system.

For this project, the main focus is not to build every feature found in a large commercial HR product. The practical goal is to understand the HR process first, identify the information that has to be stored, define who can access it, and then build a small but usable first version.

The proposed HRMS will concentrate on employee records, user authentication, attendance, leave management, login and logout tracking, and basic reporting. These functions directly match the initial requirement and are realistic to implement as the first development stage.

2. Understanding the Requirement

The requirement can be understood as a single employee-management workflow rather than as a collection of unrelated pages. An employee should have an identity in the system. That identity should be connected to the employee's department, role and other permitted information.

Once the employee has an account, the system can record login and logout activity, attendance events and leave requests. HR or an authorized manager should be able to view and manage the information according to their role.

The important requirements identified for the first version are:

- Maintain employee information such as employee ID, name, role, department and salary-related details.
- Allow authorized users to log in securely.
- Record attendance using check-in and check-out actions.
- Provide a leave request and approval process.
- Record login and logout events for work-session analysis.
- Give HR, managers and employees different levels of access.
- Provide simple dashboards and reports for useful HR information.
- Keep important changes traceable through basic audit information.

3. Proposed HRMS: What We Are Actually Building

The proposed system can be treated as an Employee Operations HRMS. Its purpose is to manage the daily employee information and activities that are needed by a small or medium-sized organization.

The first version will contain the following core areas:

- Employee Management – create, view, update and manage employee profiles.
- Authentication – login, password protection and token/session-based access.
- Role Management – employee, manager and HR/admin permissions.
- Attendance – check-in, check-out and attendance history.
- Leave Management – leave request, review, approval/rejection and history.
- Work-Session Tracking – login/logout records and session duration.
- Dashboard and Reports – attendance, leave and employee summaries.
- Audit Information – important actions can be recorded for accountability.

Features such as full payroll processing, recruitment, performance appraisal, learning management and biometric integration are not required for the first build. They can be considered after the basic system works reliably.

4. Users of the System

The same application should not expose every operation to every user. Access should depend on the user's role. This keeps the system easier to understand and also reduces the chance of unauthorized changes.

Role	Typical Access	Sensitive Operations
Employee	Own profile, attendance and leave	No access to other employees' restricted data
Manager	Team attendance and leave review	Approve/reject assigned team requests
HR / Admin	Organization-wide HR operations	Employee records, roles and reports

Role and Access Overview

Employee

- View permitted personal and employment information.
- Log in and log out.
- Mark attendance according to the organization's policy.
- Apply for leave and view leave status.
- View personal attendance and leave history.

Manager

- View employees under the manager's responsibility.
- Review attendance information where permitted.
- Review and act on leave requests.
- View basic team-level reports.

HR / Administrator

- Create and manage employee records.
- Assign roles and organizational information.
- Manage leave rules and employee leave records.
- View organization-level attendance and leave reports.
- Review audit information and manage administrative operations.

5. Employee Management

Employee management is the central part of the proposed system because attendance, leave and reporting depend on knowing which employee a record belongs to.

An employee profile can contain:

- Employee ID
- Full name
- Email and contact details

- Department
- Designation or role
- Joining date
- Employment status
- Salary-related information, where access is authorized
- User account reference
- Other organization-specific fields required by HR

The application should avoid displaying salary or other restricted information to ordinary employees unless the organization specifically permits it. The database can store the information while the API controls who is allowed to retrieve it.

6. Authentication and Login/Logout

Authentication verifies who is using the system. It is separate from authorization, which decides what that user is allowed to do.

A practical implementation can use password hashing for stored passwords and JWT-based authentication for the API. After successful login, the user receives an authenticated session or token. The token is then checked when protected API operations are requested.

A normal login flow is:

1. User enters email/username and password.
2. Backend validates the account and compares the password securely.
3. Backend creates an authenticated session/token.
4. The application records the login event.
5. User accesses only the operations allowed by the assigned role.
6. On logout, the system records the logout event and session information.

Login/logout data can help understand work-session duration, but it should not be treated as an exact measurement of employee productivity. Someone can remain logged in without actively working, and someone may perform work outside the application. Actual productivity is better evaluated using tasks, outcomes, targets or other organization-approved measures.

7. Attendance Management

Attendance is the record of whether and when an employee is present according to the organization's attendance policy. The simplest first implementation uses check-in and check-out actions.

Basic attendance flow:

1. Employee logs in.
2. Employee selects Check In.
3. The system records the date and check-in time.
4. During the working period, the attendance record remains open.
5. Employee selects Check Out.
6. The system records the check-out time and calculates the session duration where appropriate.
7. The employee and authorized HR/manager users can view the resulting attendance record.

Important cases that should be considered during development:

- An employee should not be able to create multiple active check-ins for the same working period.
- Check-out should normally require an active attendance record.
- A missed check-out should have a defined handling rule.
- Late arrival and early departure rules should be configurable if required.
- Weekends and holidays may need separate treatment.
- Manual correction should be restricted to authorized users and should be traceable.

8. Leave Management

Leave management converts an employee's leave requirement into a controlled request and approval process. The exact rules depend on the organization's leave policy, so the application should keep business rules configurable instead of assuming one universal policy.

Typical leave workflow:

1. Employee selects the leave type and requested dates.
2. Employee provides a reason or supporting information if required.
3. System validates basic rules such as date order and available leave balance when applicable.
4. Request is stored with a Pending status.
5. Authorized manager or HR reviews the request.
6. The request becomes Approved or Rejected.
7. If approved, the leave balance is updated according to the organization's rule.
8. The employee can view the final status.

Possible leave types include:

- Casual leave
- Sick leave
- Earned/annual leave
- Permission or other organization-specific categories

9. Work-Time Analysis and Productivity

The requirement to record login and logout times needs careful interpretation. The HRMS can calculate the duration of an application session and compare attendance with login/logout activity, but this should be considered work-time information rather than a complete productivity score.

For example, if an employee logs in at 9:05 AM and logs out at 5:45 PM, the system can calculate the session duration. This is useful for attendance and operational reporting, but it does not prove how much productive work was completed.

A better future productivity module could combine approved task completion, project progress, targets or performance-review information. Any such feature should be designed transparently and according to company policy rather than using login duration alone.

10. How the Main Workflow Connects

The proposed application can be viewed as one connected flow:

User Account → Authentication → Employee Profile → Attendance / Leave → Manager or HR Review → Dashboard / Reports → Audit Information

This connection is important because it avoids creating separate systems for each HR activity. The employee identity becomes the common reference for attendance, leave and other permitted records.

11. Information and Database Design

A relational database is suitable because HRMS information has clear relationships. An employee belongs to a department, has a user account, can have many attendance records and can submit multiple leave requests.

For the first version, the main database entities can be:

- users – login credentials, account status and authentication-related information.
- employees – employee profile and organization details.
- roles – defines access categories.
- attendance – check-in, check-out and attendance status.
- leave_requests – requested dates, leave type, reason and approval status.
- audit_logs – important administrative or system actions.
- departments – organization structure if required by the project.
- leave_types – organization-defined leave categories.

The database should use primary keys for unique records and foreign keys to connect related information. For example, an attendance record should point to the employee it belongs to rather than storing an unrelated name as the only reference.

12. Proposed Technical Architecture

The project can be implemented as a web application with a frontend, backend API and relational database. Since the team is already working with FastAPI and PostgreSQL, those technologies are a practical choice for this project.

Suggested technology choices:

- Backend: FastAPI with Python
- Database: PostgreSQL
- ORM/database access: SQLAlchemy
- Authentication: JWT-based authentication with secure password hashing
- API documentation/testing: Swagger UI through OpenAPI
- Frontend: React or another simple web frontend used by the team
- Development tools: VS Code, Git and GitHub
- Database administration: pgAdmin or equivalent PostgreSQL tool

The request flow can be represented as:

Browser / Frontend → FastAPI API → Authentication & Business Logic → SQLAlchemy → PostgreSQL

This layered approach keeps the user interface, business rules and data storage separate. It also makes the application easier to test and extend.

13. API-Level Implementation Plan

The backend can be divided into logical API groups rather than placing all operations in one file.

- Authentication: register user, login, logout/session handling, current-user information.

- Employees: create employee, list employees, view one employee, update employee, deactivate employee.
- Attendance: check-in, check-out, today's attendance, attendance history and authorized reports.
- Leave: create request, list personal requests, approve/reject request, leave history and balance where applicable.
- Reports: attendance summary, leave summary and employee-level or team-level statistics.
- Administration: role/department management and selected audit information.

The exact endpoint names can be finalized after the database schema and frontend workflow are agreed. The important point is to keep the API organized by business responsibility.

14. Security and Privacy Considerations

HRMS data is sensitive because it can contain employee identity information, salary-related information, attendance and leave records. Security therefore has to be considered from the first version rather than added at the end.

- Never store plain-text passwords; use secure password hashing.
- Protect private endpoints using authentication.
- Apply role-based authorization on the server, not only in the frontend.
- Do not expose salary or administrative information to unauthorized users.
- Store secret keys and database credentials in environment variables.
- Use HTTPS in a deployed environment.
- Validate request data on the backend.
- Record important administrative changes in audit information.
- Use database backups and recovery procedures.
- Avoid collecting personal information that the application does not actually need.
- Define who can view, edit and export employee information.

For an actual organizational deployment, privacy, employment, tax and labour requirements should be reviewed with the organization's responsible HR/legal team. Building the software does not automatically make the organization legally compliant.

Technical Stack Summary

Entity	Stores	Relationship / Purpose
users	Login and account information	Connects a system user to an employee
employees	Employee profile	Central employee record
roles	Access roles	Controls permitted operations
attendance	Check-in/out records	Linked to employee
leave_requests	Leave applications and status	Linked to employee and reviewer
departments	Organization units	Groups employees
leave_types	Leave categories	Defines available leave types
audit_logs	Important actions	Provides traceability

Core Database Entities

Layer	Proposed Technology	Reason
Backend API	FastAPI / Python	Suitable for REST APIs, validation and modular backend development
Database	PostgreSQL	Relational structure and reliable transactions
ORM	SQLAlchemy	Connects application models with

		PostgreSQL
Authentication	JWT + password hashing	Secure API access and role-based authorization
API Testing	Swagger UI / OpenAPI	Easy testing and documentation during development
Frontend	React or simple web UI	Provides employee, manager and HR screens

15. Feasibility Study

Technical Feasibility

The project is technically feasible. The required operations are standard CRUD and workflow operations, and FastAPI with PostgreSQL can support them. Authentication, role-based access, attendance records and leave workflows can be implemented incrementally.

Operational Feasibility

The system is practical if the organization first defines clear rules for attendance, leave approval, employee roles and access permissions. Without clear HR policies, software may record data correctly but still produce confusing results.

Financial Feasibility

A basic internal HRMS can be developed using open-source technologies. The major costs in a real deployment would come from development effort, hosting, backups, maintenance, security and optional integrations rather than from the core frameworks.

Schedule Feasibility

A limited first version is feasible for an initial development sprint if the team focuses on authentication, employee management, attendance, leave and a basic dashboard. Trying to include payroll, recruitment, performance management and biometric integration immediately would increase complexity and risk.

Security Feasibility

Security is feasible, but it requires deliberate implementation. Password hashing, authentication, authorization, input validation, secret management, audit records and database backups should be treated as core development tasks.

16. Major Challenges to Expect

- Attendance edge cases such as forgotten check-out, duplicate actions and manual corrections.
- Different leave policies for different employee groups.
- Permission mistakes that allow users to see information they should not see.
- Handling salary-related data with stronger access restrictions.
- Keeping reports accurate when attendance or leave records are corrected.
- Maintaining a clear audit trail for administrative changes.
- Migrating existing employee data if the organization already uses spreadsheets or another system.
- Making the interface simple enough for employees and HR staff to use without extensive training.

The most important development principle is to finalize business rules before writing too much code. A technically correct application can still fail if the HR rules are unclear.

17. Development Approach

The recommended approach is to build the system in small working stages.

Stage	Main Work	Expected Result
1	Foundation, database and project structure	Running FastAPI + PostgreSQL base
2	Authentication and employee management	Secure user access and employee CRUD
3	Attendance	Working check-in/check-out and history
4	Leave management	Request and approval workflow
5	Dashboard, audit and testing	Usable MVP with basic reports

Suggested Initial Development Sequence

Stage 1 – Foundation

- Finalize requirements and user roles.
- Create the PostgreSQL database.
- Design tables and relationships.
- Set up the FastAPI project structure.
- Implement configuration and database connection.

Stage 2 – Authentication and Employee Management

- Create registration or controlled user creation.
- Implement login and secure password handling.
- Implement role-based access.
- Create employee CRUD operations.
- Test protected API routes using Swagger UI.

Stage 3 – Attendance

- Implement check-in and check-out.
- Prevent duplicate active attendance.
- Store attendance history.
- Add basic attendance calculations and reports.

Stage 4 – Leave

- Create leave types and requests.
- Implement pending, approved and rejected states.
- Add manager/HR approval.
- Add leave history and balance logic where required.

Stage 5 – Dashboard and Testing

- Create employee, manager and HR views.
- Display basic attendance and leave summaries.
- Add audit information for important actions.
- Perform API, validation, authorization and workflow testing.

18. Testing Strategy

Testing should not only check whether an endpoint returns a response. HRMS testing should also verify that the right user can perform the right operation and that invalid actions are blocked.

Important test areas:

- Login with valid and invalid credentials.
- Access a protected endpoint without authentication.
- Employee attempts an HR-only operation.
- Create, update and deactivate an employee.
- Check in twice without checking out.
- Check out without an active attendance record.
- Submit a valid and invalid leave request.
- Approve or reject a request using an authorized account.
- Check that employees cannot view another employee's restricted information.
- Verify that reports match stored attendance and leave records.
- Verify that important administrative changes are recorded.

19. Future Scope

Once the core HRMS is stable, the system can be expanded without changing the basic employee identity model.

- Payroll processing and salary history.
- Employee document storage.
- Holiday and shift management.
- Onboarding and offboarding workflows.
- Recruitment and applicant tracking.
- Performance management and goal tracking.
- Learning and training management.
- Email or notification integration.
- Biometric or device-based attendance integration.
- Advanced workforce analytics.
- Integration with accounting or payroll systems.
- Mobile application support.

These features should be treated as later phases. The first objective is to build a reliable foundation for employee identity, authentication, attendance and leave.

20. Final Feasibility Conclusion

Based on the functional requirements and the proposed implementation approach, the HRMS project is feasible as a web-based application. The core modules can be developed using technologies already suitable for the team, particularly FastAPI and PostgreSQL.

The most realistic first version is not a complete enterprise HR suite. It is a focused employee operations system that handles employee records, authentication, attendance, leave and basic reporting. This keeps the initial development manageable while leaving a clear path for future expansion.

The main risk is not the technology itself. The bigger risk is implementing unclear HR rules or giving users incorrect access to sensitive information. Therefore, requirements, roles, approval rules and data permissions should be finalized before the full development begins.

Recommended decision: proceed with development of the MVP, validate the workflow with HR/management requirements, test each module independently, and add advanced HR features only after the core system is stable.

21. References and Research Areas

The following sources can be used for further study and validation while developing the project. They are reference points rather than content to be copied into the implementation.

- SAP – Human Capital Management / HR technology concepts
- Workday – Human Capital Management concepts and employee lifecycle workflows
- Zoho People – practical HRMS module examples
- OWASP – Web application security and authentication guidance
- India Code – Digital Personal Data Protection Act, 2023
- MeitY – digital and data-protection related information
- Ministry of Labour & Employment, Government of India – labour and employment information
- Income Tax Department, Government of India – salary and tax-related information

For production deployment, the organization should verify the latest legal, privacy, labour and tax requirements with appropriate professional guidance instead of relying only on this learning document.